# Cloud Computing TASK – 2

1. **List three popular apps you use (like Instagram, Zomato, or Spotify) and for each, identify whether it is most similar to IaaS, PaaS, or SaaS. Briefly explain your reasoning for each choice.**

   **Ans :**

| Application | Cloud Service Model | Reason |
|---|---|---|
| Instagram | SaaS (Software as a Service) | Instagram is a complete application that users access through a mobile app or web browser without installing or managing any servers or software. |
| Spotify | SaaS (Software as a Service) | Spotify provides music streaming as a ready-to-use online service. Users simply sign in and listen to music without managing the underlying infrastructure. |
| Google App Engine | PaaS (Platform as a Service) | Google App Engine provides a platform where developers can build, deploy, and manage applications without handling servers or operating systems. |

2. **Create a table comparing IaaS, PaaS, and SaaS with at least three points for each model, using real-world examples such as AWS EC2, Google App Engine, and Office 365.**

   **Ans :**

| Feature | IaaS (Infrastructure as a Service) | PaaS (Platform as a Service) | SaaS (Software as a Service) |
|---|---|---|---|
| What it provides | Virtual servers, storage, and networking infrastructure | A platform for developing, testing, and deploying applications | Ready-to-use software delivered over the internet |
| User manages | Operating system, applications, and data | Applications and data | Only user settings and data |
| Provider manages | Physical hardware, networking, and virtualization | Hardware, operating system, runtime, and infrastructure | Everything, including hardware, software, updates, and maintenance |
| Best for | Organizations needing full control over infrastructure | Developers building and deploying applications | End users who need software without installation |
| Real-world example | AWS EC2 | Google App Engine | Microsoft Office 365 |

3. **Sign up for a free AWS account (or use an existing one), navigate to the EC2 dashboard, and launch a new t2.micro EC2 instance running Ubuntu 22.04. Take a screenshot of your**

**running instance in the AWS console.<br><br><em><strong>Hint:</strong> Use the 'Launch Instance' button and select the free tier eligible Ubuntu AMI.</em>**

**Ans :**

- Logged in to the AWS Management Console.
- In the search bar, typed EC2 and opened the EC2 Dashboard.
- Clicked the Launch Instance button.
- Entered an instance name (e.g., Ubuntu-Server).
- Selected Ubuntu Server 22.04 LTS as the Amazon Machine Image (AMI).
- Chose t2.micro as the instance type (Free Tier eligible).
- Created a new key pair (or selected an existing one) and downloaded the .pem file.
- Kept the default network settings and allowed SSH (port 22).
- Clicked Launch Instance.
- Waited until the instance state changed to Running.
- Opened the Instances page and verified that the EC2 instance was running.

4. **After your EC2 instance is running, connect to it using SSH from your local terminal or an online SSH client. Run the command 'uname -a' and note the output.<br><br><em><strong>Hint:</strong> You may need to use the .pem key file downloaded during instance setup.</em>**

   **Ans :**

   - Opened the local terminal.
   - Navigated to the folder containing the downloaded .pem key file.
   - Changed the key file permissions using:

- chmod 400 my-key.pem
- Connected to the EC2 instance using SSH:
- ssh -i my-key.pem ubuntu@<Public-IP-Address>
- After successfully logging in, ran the following command:
- uname -a

5. **Suppose you want to host a simple static website for your college fest using your EC2 instance. List the basic steps you would follow after launching the instance to make your site accessible to everyone. Write 3-5 steps in order**.

   **Ans :**

   - Connect to the EC2 instance using SSH and install a web server such as Apache or Nginx.
   - Upload the website files (HTML, CSS, JavaScript, and images) to the web server's root directory.
   - Configure the EC2 security group to allow inbound HTTP traffic on port 80 (and HTTPS on port 443 if required).
   - Start the web server and test the website using the EC2 instance's public IP address.